ACTIVIDAD

Dosier de Actividades del Tema: Estructuras de datos avanzadas

Desarrollo de Aplicaciones Web/ Desarrollo de Aplicaciones Multiplataforma/Administración de Sistemas Informáticos y Redes

Programación

Linkia FP

Se debe entregar un zip que contenga todos los archivos. java que has creado. Para poder aprobar un ejercicio, éste debe poder ejecutarse sin errores.

En esta actividad, se te solicita implementar un programa en Java que permita gestionar una lista de productos utilizando las clases ArrayList y HashMap. Se debe implementar lo siguiente:

EJERCICIO 1: Crea una clase llamada Producto con los siguientes atributos: nombre (String), ArrayList<String> materiales, precio (double) y cantidad (int). Además, implementa los métodos getters, setters y un constructor.✅

```
package DAW_M03_ACT10_T10;

import java.util.ArrayList;

/**
 *
 * @author cristiancantero
 */
public class Producto {
    private String nombre;
    private ArrayList<String>materiales;
    private double precio;
    private int cantidad;

    public Producto() {

    }

    public Producto(String nombre, ArrayList<String> materiales, double precio, int cantidad) {
        this.nombre = nombre;
        this.materiales = materiales;
        this.precio = precio;
        this.cantidad = cantidad;
```

```
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public ArrayList<String> getMateriales() {
    return materiales;
}

public void setMateriales(ArrayList<String> materiales) {
    this.materiales = materiales;
}

public double getPrecio() {
    return precio;
}

public void setPrecio(double precio) {
    this.precio = precio;
}

public int getCantidad() {
    return cantidad;
}

public void setCantidad(int cantidad) {
```

```
        this.cantidad = cantidad;
    }

    @Override
    public String toString() {
        return "Producto{" + "nombre=" + nombre + ", materiales=" + materiales + ", precio=" + precio
+ ", cantidad=" + cantidad + '}';
    }
```

EJERCICIO 2: Sobreescribe los métodos equals() y hashCode() en la clase Producto. La comparación debe basarse en el nombre del producto.✅

```
@Override
    public int hashCode(){
        int result = this.nombre.hashCode();
        return result;
    }

    @Override
    public boolean equals(Object obj){
        if(this==obj)  ==>Si el primer objeto es como el segundo
            return true; ==> Está dentro (son iguales)
        if(obj == null) ==> Si no
            return false; ==> Está fuera (no son iguales)
        if(getClass() != obj.getClass()) ==> Si no son del mismo tipo
            return false; ==> Está fuera

            Producto other = (Producto) obj; Convierto (Producto)obj a tipo Producto para poder
usarlo(cast)
```

```
    if(this.nombre==null){ ==> Si este nombre esta vacío
      if(other.nombre !=null) ==> Y este también
        return false; ==> Está fuera

    }else if(!nombre.equals(other.nombre)) ==> Si el nombre no es igual al otro nombre
      return false; ==> Está fuera.
    return true; ==> Si llegó hasta aquí los dos objetos son iguales
  }



}
```

Voy a documentar mi periplo con equals(). He tenido una pelea muy fuerte porque me costó llegar al punto de entender que ***this*** es el objeto uno y que ***obj*** es el objeto dos. Y que comparamos this con obj. Esto empieza a suceder desde (this==obj) pero a partir de la linea Producto other = (Producto) obj; tengo acceso al atributo. Al convertir other en un objeto Producto y poder usarlo puedo compararlo con el primer objeto (this). He documentado cada línea de código en método equals() para tener clara la ruta.

EJERCICIO 3: Dentro del método main, crea un HashMap<String, Producto> listaProductos, donde la clave será el código del producto y el valor será un objeto de tipo Producto. ✅

```
public class Main {


  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    Funciones f = new Funciones();
    Producto p = new Producto();
    HashMap<String, Producto> listaProductos = new HashMap<>();
    ArrayList<String> materiales = new ArrayList<>();
```

Es el principio del Main. Ya se ven declarados Scanner y la clase Funciones instanciada para poder llamar a las funciones desde el método.

EJERCICIO 4: Implementa un menú que permita al usuario realizar las siguientes opciones:✅

1. Agregar producto: Solicita al usuario ingresar el nombre, precio y cantidad del producto. Verifica si el producto ya existe en la lista utilizando los métodos equals() y hashCode(). Si el producto ya existe, muestra un mensaje "El producto ya existe". De lo contrario, crea un objeto Producto con los datos ingresados y agrégalo a listaProductos.

2. Actualizar producto: Solicita al usuario ingresar el nombre del producto a actualizar. Verifica si el producto existe en la lista. Si existe, solicita al usuario ingresar el nuevo precio y cantidad del producto y actualiza los valores correspondientes. Si no existe, muestra un mensaje "El producto no existe".

3. Eliminar producto: Solicita al usuario ingresar el nombre del producto a eliminar. Verifica si el producto existe en la lista. Si existe, elimina el producto de listaProductos. Si no existe, muestra un mensaje "El producto no existe".

4. Salir: Finaliza la ejecución del programa.

Dejo todo el método main:

```
package DAW_M03_ACT10_T10;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;

/**
 *
 * @author cristiancantero
 */
public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Funciones f = new Funciones();
        Producto p = new Producto();
```

```
HashMap<String, Producto> listaProductos = new HashMap<>();
ArrayList<String> materiales = new ArrayList<>();
boolean valido = false;


do {
   try {
      System.out.println("====Menú====");
      System.out.println("1. Agregar producto");
      System.out.println("2. Actualizar producto");
      System.out.println("3. Eliminar producto");
      System.out.println("0. Salir");
      System.out.println("=============");
      int opcion = sc.nextInt();
      sc.nextLine();

      switch (opcion) {
         case 1:
            f.ingresarProducto(listaProductos, p);
            break;
         case 2:
            f.actualizarProducto(sc, listaProductos);
            break;

         case 3:
            f.eliminarProducto(sc, listaProductos);
            break;
         case 0:
            System.out.println("Adios");
            valido = true;
            break;
```


Linkia FP

```
                default:
                    System.out.println("Introduce una opción válida");
            }
        } catch (Exception e) {
            System.out.println("Error al validar");
        }
    } while (!valido);

    }
}
```

He creado la clase Funciones porque a nivel personal prefiero separar responsabilidades de esta forma. Dejo aquí el código de toda la clase con todas las funicones.

```
package DAW_M03_ACT10_T10;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
import java.util.Scanner;

/**
 *
 * @author cristiancantero
 */
public class Funciones {

    private HashMap<String, Producto> listaProductos = new HashMap<>();
    private ArrayList<String> materiales = new ArrayList<>();


    public Funciones(){

    }
```

```
public void ingresarProducto(HashMap<String,Producto>listaProductos, Producto p){
    boolean existe = false;
   for(Producto pro : listaProductos.values()){

      if(pro.equals(p)){

         existe = true;
         break;
      }
   }
   if(existe){
      System.out.println("El producto ya existe");
   }else{
      String codigo = "P" + (listaProductos.size()+1);
      listaProductos.put(codigo,p);
      System.out.println("Producto añadido con código: " + codigo);
   }
   System.out.println("====Lista de productos====");
   for(Producto pr : listaProductos.values()){
      System.out.println(pr);
   }
}

public void agregarProducto(Scanner sc) {

   Producto p = new Producto();
   boolean mats = false;
   System.out.println("Introduce el nombre del producto: ");
   String nombre = sc.nextLine();
   p.setNombre(nombre);
```

```
do {
    try {
        System.out.println("Introduce el material");
        String material = sc.nextLine();
        materiales.add(material);

        System.out.println("¿Quieres introducir más materiales? S/N");
        String introducir = sc.nextLine();
        if (introducir.equalsIgnoreCase("s")) {
            mats = false;
        } else {
            mats = true;
        }

    } catch (Exception e) {
        System.out.println("Ingresa una opcion válida");
    }
} while (!mats);
System.out.println("Introduce el precio");
double precio = sc.nextDouble();
sc.nextLine();
p.setPrecio(precio);
System.out.println("Introduce la cantidad");
int cantidad = sc.nextInt();
sc.nextLine();
p.setCantidad(cantidad);

ingresarProducto(listaProductos, p);
System.out.println("====Lista productos====");
for(Producto pr: listaProductos.values()){
    System.out.println(pr);
```

```
    }

}

public void eliminarProducto(Scanner sc, HashMap<String,Producto>listaProductos){
    System.out.println("Introduce el nombre a eliminar: ");
    String nombreEliminar = sc.nextLine();

    String claveAEliminar = null; //es null porque la clave del hashmap es un String

    for(String clave :listaProductos.keySet()){
        Producto product=listaProductos.get(clave);

        if(product.getNombre().equalsIgnoreCase(nombreEliminar)){
            claveAEliminar = clave;
            break;
        }
    }
    if(claveAEliminar != null){
        listaProductos.remove(claveAEliminar);
        System.out.println("Producto eliminado");
    }else{
        System.out.println("No se encontró el producto");
    }
    System.out.println("====Lista productos====");
    for(Producto pr: listaProductos.values()){
        System.out.println(pr);
    }
}

public void actualizarProducto(Scanner sc, HashMap<String,Producto> listaProductos){
```


Linkia FP

```
    System.out.println("Introduce el producto que quieres modificar");
    String nombreModificar=sc.nextLine();

    boolean encontrado = false;

    for(Producto pro : listaProductos.values()){
        if(pro.getNombre().equalsIgnoreCase(nombreModificar)){

            System.out.println("Introduce el nuevo precio: ");
            double nuevoPrecio = sc.nextDouble();
            sc.nextLine();
            pro.setPrecio(nuevoPrecio);

            System.out.println("Introduce la nueva cantidad: ");
            int nuevaCantidad = sc.nextInt();
            sc.nextLine();
            pro.setCantidad(nuevaCantidad);

            encontrado = true;
            break;
        }
    }
    if(!encontrado){
        System.out.println("No se encuentra el producto");
    }

    System.out.println("====Lista productos====");
    for(Producto pr: listaProductos.values()){
        System.out.println(pr);
    }
}
```

}

La clase ***Funciones*** empeza declarando el HashMap con String como clave y Producto como valor. Dentro de Producto como atributo tenemos un ArrayList, así que habrá que tratarlos a cada uno con sus métodos.

Para guardar el producto he utilizado 2 funciones. Una que recoje todos los datos introducidos por el user y otra que guarda todos esos datos dentro del HashMap.

IngresarProducto() Recibe como parámetros el HashMap y un objeto de tipo Producto.

Empiezo declarando una bandera booleana que me servirá más adelante para validar si el producto ya existe. Recorro con un for/each el HashMap usando el método values() que devuelve una colección con los valores del HashMap. Si hay coincidencia, la bandera booleana pasa a true. Eso significa que el producto existe entonces manda un mensaje diciendolo. Si no existiera, hay que almacenarlo. Declaro una variable para guardar el código ID generado, lo guardo en el HashMap usando el método .put y lanzo un mensaje de guardado correctamente.

El método acaba refrescando la lista de productos tal y como manda el ejercicio.

Después de realizar cada opción del menú, muestra por pantalla los elementos de listaProductos.

agregarProducto() Es la función que se va a encargar de recoger los datos del usuario, pushearlos al objeto y finalmente usará el método ingresarProducto() para hacer la inserción en el HashMap. La forma es bastante parecida a la de siempre. Se pregunta al usuario, se guarda en una variable y y esa variable se añade al objeto usando los setters. Cuando he tenido que trabajar con el ArrayList materiales, he implementado una lógica de bucle por si el usuario quiere introducir más de un material. Al final usamos ingresarProducto() para pasarle al HashMap los datos y refrescamos el stock para que se vea.

eliminarProducto() recibe como parámetros un objeto Scanner y el HashMap listaProductos. Se pregunta al user el nombre del producto a eliminar. Se guarda en una variable. Inicializo una varialbe que será la claveAEliminar. La inicializo en null porque la clave de mi HashMap es un String. Esto puede cambiar según la clave del HashMap con el que estemos trabajando. Recorro con un for/each la llista de productos usando el método keySet() que devuelve una colección con las claves del HashMap. En cada iteración guardo la clave en una variable. Si esa variable coincide con el nombre que se quiere eliminar se almacena en claveAEliminar. De esta forma, claveAEliminar deja de ser null. Esto nos servirá para la continuar con la función. Si claveAEliminar NO es null usamos el método .remove para eliminarla. Se muestran los mensajes y se refresca el stock.

actualizarProducto() Esta función la vamos a usar para cambiar el precio del producto si el user desea hacerlo. Recibe como parámentros un objeto Scanner y el HashMap. Se le pregunta al usuario el nombre del producto a modificar. Se guarda. Declaro bandera booleana para controlar acciones. Recorro con un for/each el hashmap usando el metodo .values(). Usando los getters accedo al nombre y con equals si coincide con el nombre almacenado, se le pide al user que introduzca el nuevo precio y la nueva cantidad. Usando los setters se accede al objeto y se cambia.

La bandera booleana pasa a true. Si esta bandera no hubiera sido true no se habría encontrado el producto. Se lanza mensaje para que se sepa. Y se acaba actualizando el stock.

==========

BLOQUE DEL OUTPUT. A continuación documento el funcionamiento del programa.

```
run:
====Menú====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
0. Salir
=============
1
Introduce el nombre del producto:
Mesa
Introduce el material
madera
¿Quieres introducir más materiales? S/N
s
Introduce el material
hierro
¿Quieres introducir más materiales? S/N
n
Introduce el precio
400
Introduce la cantidad
2
Producto añadido con código: P1
====Lista de productos====
Producto{nombre=Mesa, materiales=null, precio=400.0, cantidad=2}
====Lista productos====
Producto{nombre=Mesa, materiales=null, precio=400.0, cantidad=2}
====Menú====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
0. Salir
=============
0
Adios
BUILD SUCCESSFUL (total time: 33 seconds)
```

Empiezo usando la primera opción. Introduce bien los datos, pero a la hora de refrescarlos materiales=null y repite 2 veces la lista de productos. Hay que investigarlo.

Pues una vez investigado.. El stock se repetía dos veces porque tanto in ingresarProducto() como en agregarProducto() tenían el refresco del stock al final de la función. Ya lo arregle eliminando uno de los dos. A parte de usar el metodo .add para los materiales también debía setearlo p.setMateriales(materiales). Eliminé hashmap como atributo de clase y lo incorporé a la función porque resulta que si no todos compartían la misma lista (todo esto me va pillando a traspiés).

Así que volvemos al output:

```
run:
====Menú====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
0. Salir
=============
1
Introduce el nombre del producto:
mesa
Introduce el material
madera
¿Quieres introducir más materiales? S/N
s
Introduce el material
patas de hierro
¿Quieres introducir más materiales? S/N
n
Introduce el precio
200
Introduce la cantidad
3
Producto añadido con código: P1
====Lista de productos====
Producto{nombre=mesa, materiales=[madera, patas de hierro], precio=200.0, cantidad=3}
====Menú====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
0. Salir
=============
2
Introduce el producto que quieres modificar
mesa
No se encuentra el producto
====Lista productos====
====Menú====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
0. Salir
=============
```


Linkia FP

No se encuentra el producto… Toca volver a investigar.

El problema se encontraba en los parámetros… si le paso por parametros un hashmap no uso el hashmap de clase, es otro nuevo y cuando quiero actualizar no encuentra lo que busca poruqe no está en ese hashmap, está en otro que me he inventado por parámetros. He eliminado todos los hashmap por parámetros que tenia. Así que de esta manera queda el output:

```
DAW_M03_T10_ACT10_CristianCantero (run) ×   DAW_M03_T10_ACT10_CristianCantero (run) #2 ×
run:
====Menú====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
0. Salir
=============
1
Introduce el nombre del producto:
mesa
Introduce el material
madera
¿Quieres introducir más materiales? S/N
s
Introduce el material
hierro
¿Quieres introducir más materiales? S/N
n
Introduce el precio
200
Introduce la cantidad
3
Producto añadido con código: P1
====Lista de productos====
Producto{nombre=mesa, materiales=[madera, hierro], precio=200.0, cantidad=3}
====Menú====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
0. Salir
=============
2
Introduce el producto que quieres modificar
mesa
Introduce el nuevo precio:
100
Introduce la nueva cantidad:
2
====Lista productos====
Producto{nombre=mesa, materiales=[madera, hierro], precio=100.0, cantidad=2}
====Menú====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
0. Salir
=============
3
Introduce el nombre a eliminar:
mesa
Producto eliminado
====Lista productos====
====Menú====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
0. Salir
=============
0
Adios
BUILD SUCCESSFUL (total time: 37 seconds)
```

El programa a agregado, actualizado y eliminado correctamente el producto como se puede ver.

CONCLUSIONES: Trabajar con ArrayList y con HashMap (sobre todo el segundo) requiere re-revisar todo. Si que sirve lo que he estudiado antes pero, a bote pronto, parece que esté haciendo otra cosa totalmente diferente.

Cosas arreglar: Tengo que tener más presente los atributos de clase y las variables de entorno. Muchas veces se me olvida que tengo eso ahí arriba. Tengo que poner más foco en los patrones que se repiten. Y, como no, **hay que estudiar mucho.**

Bibliografía utilizada: Imprimibles Linkia (sufrimiento), chatGPT (salvación).

Recuerda utilizar los métodos adecuados de ArrayList y HashMap para agregar, actualizar y eliminar elementos. Utiliza los métodos equals() y hashCode() para comparar los productos.

Ejemplo de resultado

```
===== MENÚ =====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
4. Salir
Ingrese una opción: 1
===== AGREGAR PRODUCTO =====
Ingrese el nombre del producto: Mesa
Ingrese el precio del producto: 120
Ingrese la cantidad del producto: 1
Ingrese los materiales del producto (ingrese 'fin' para terminar):
Madera
Tornillería
Cristal
fin
Producto agregado exitosamente
===== LISTA DE PRODUCTOS =====
Producto{nombre='Mesa', materiales=[Madera, Tornillerıa, Cristal], precio=120.0, cantidad=1}
```

```
===== MENÚ =====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
4. Salir
Ingrese una opción: 2
===== ACTUALIZAR PRODUCTO =====
Ingrese el nombre del producto a actualizar: Mesa
Ingrese el nuevo precio del producto: 342
Ingrese la nueva cantidad del producto: 2
Producto actualizado exitosamente
===== LISTA DE PRODUCTOS =====
Producto{nombre='Mesa', materiales=[Madera, Tornillerda, Cristal], precio=342.0, cantidad=2}

===== MENÚ =====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
4. Salir
Ingrese una opción: 3
===== ELIMINAR PRODUCTO =====
Ingrese el nombre del producto a eliminar: Mesa
Producto eliminado exitosamente
===== LISTA DE PRODUCTOS =====


===== MENÚ =====
1. Agregar producto
2. Actualizar producto
3. Eliminar producto
4. Salir
Ingrese una opción: 4
Saliendo del programa...
===== LISTA DE PRODUCTOS =====

BUILD SUCCESSFUL (total time: 1 minute 28 seconds)
```


Linkia FP